JSOLV: Java Static Object Labelling for Verification Application-Owned Value-State Contracts for Java

Jayanta Majumder Sambuddha Majumder

10 October 2026

Abstract

An ordinary Java type often says little about whether a value has passed through the workflow stages required by its consumer. A document may have been persisted, validated, and authenticated while remaining an ordinary document throughout. JSOLV, "Java Static Object Labelling for Verification", implements a lightweight layer for expressing and checking such facts. Application programmers declare their label vocabulary in a two-key configuration file. A Maven plugin generates the annotation classes before compilation; a generic annotation processor recognises their shared marker and checks requirements at call sites. Return annotations establish facts, parameter annotations require them, assignments replace tracked facts, and branch joins retain their intersection. An explicit preservation contract transfers the known labels of one input to a result, allowing independent workflow stages to compose without enumerating label combinations. This paper presents the implementation, its set-based semantics, a complete usage example, build integration, diagnostic reporting, and a small regression evaluation. It also distinguishes contract consistency from semantic truth: annotated returns and preservation are trusted, and the prototype does not provide sound analysis of arbitrary Java mutation, aliasing, or control flow. JSOLV is a concrete, deliberately restricted sequel to proposals for user-defined static property tags and for separating transient knowledge from nominal type identity.

1 From a language proposal to a working tool

Two earlier papers motivate this work. *User-Defined Static Property Tags for Mainstream Programming Languages* proposes a flow-sensitive layer of symbolic facts over values [1]. *Explicit Semantic Categories for More Legible Mainstream Programming Languages* distinguishes what a value is from what is currently known about it, and argues against encoding every transient fact as a new nominal class [2].

This paper can be read independently of those proposals. Its central question is practical: can an application team manage its own vocabulary of facts, express requirements in familiar Java signatures, and obtain useful compilation errors without editing an analyser or learning a new programming language?

The language-specific name distinguishes this Java implementation from possible future implementations of the same approach for other languages. The conceptual label algebra is independent of Java, while the present generator, annotations, compiler integration, and deployment are specifically Java and Maven mechanisms.

JSOLV answers with a configuration-driven annotation generator and a generic checker. The name expands to *Java Static Object Labelling for Verification*. Here "verify" means checking that required labels follow from the supported value-flow rules and declared contracts. It does not mean proving the domain truth of authentication, persistence, or validation.

The tool's contribution is an engineering design with four parts:

1. an application-owned label vocabulary, independent of checker code;
2. a stable marker protocol connecting generated annotations to analysis;
3. a small set algebra with an explicit, label-polymorphic preservation rule;
4. ordinary Maven deployment, compiler diagnostics, and a machine-readable report.

These mechanisms are implemented. More ambitious ideas from the earlier papers, including negative properties, alias-aware revocation, and verified postconditions, remain outside this prototype.

2 The problem: an ordinary type, several independent facts

Consider an `Article`. Its nominal type describes its data and operations. The same value may also carry workflow facts: `Persisted`, `Validated`, and `Authenticated`. A publication operation may require all three. Conventional Java signatures cannot distinguish a draft from a suitably processed value unless programmers introduce wrappers, distinct types, or a separate static analysis.

Nominal wrappers can be appropriate when states represent different domain concepts or APIs. However, n independent binary facts admit up to 2^n combinations. A vocabulary of independent labels represents those combinations without requiring a corresponding family of wrapper classes. JSOLV retains the base type and overlays the facts:

```
void publish(@Persisted @Validated @Authenticated Article article) {
    // Publication logic.
}
```

At a checked call site, every required label must be known for the actual argument. Having one label does not imply another. Their domain meanings are chosen by the application; the checker treats them as atomic names.

The emphasis is on facts about values currently held by bindings. Reassigning a variable does not retain facts about its former value. Nor does a label create a runtime field, a wrapper object, or a new Java subtype. Annotation metadata is retained in class files for compilation of consumers, but no runtime checks are inserted by the checker.

3 Vocabulary ownership and generation

The application declares its vocabulary in `jsolv.properties`, beside its `pom.xml`:

```
package=org.jsolv.example.tags
labels=Persisted, Validated, Authenticated
```

The package is application-owned; it need not begin with `org.jsolv`. Labels are case-sensitive, comma-separated Java annotation names. The file uses UTF-8 Java properties syntax, including comments and continuation lines. To add `Approved`, a programmer appends the name and rebuilds. No checker registry, processor source change, or annotation-library release is needed.

The generator plugin, `org.jsolv:jsolv-maven-plugin`, runs its `generate` goal during Maven's `generate-sources` phase. It creates one annotation per label under `target/generated-sources/jsolv` and registers that directory as a compile source root. Generation precedes compilation in the ordinary Maven lifecycle [5].

For example, the generated `Validated.java` has the following essential content; fully qualified names avoid collisions with application label names:

```
package org.jsolv.example.tags;

@org.jsolv.StaticTag
```

```
@java.lang.annotation.Target({
    java.lang.annotation.ElementType.TYPE_USE,
    java.lang.annotation.ElementType.PARAMETER,
    java.lang.annotation.ElementType.METHOD})
@java.lang.annotation.Retention(java.lang.annotation.RetentionPolicy.CLASS)
public @interface Validated {}
```

The `@StaticTag` meta-annotation identifies a label to the checker. The processor discovers this marker through compiler annotation metadata; it does not read the configuration file. Vocabulary generation and contract checking are thus connected through compiled Java declarations, not through two copies of a registry.

The generator rejects missing or unknown keys, duplicate keys or labels, empty comma-separated entries, Java keywords, restricted type names, and invalid identifiers. It validates the complete configuration before changing output. Generated files carry an ownership header. Obsolete owned sources are removed; unowned sources are retained, and overwriting an unowned destination fails. Unchanged generated sources retain their timestamps.

Removing a source file does not necessarily remove its previously compiled class. After deleting or renaming labels, programmers should run `mvn clean compile`. Generated sources and classes are build artifacts; the configuration and handwritten application code are the maintained inputs.

4 Three contracts and one composition rule

4.1 Establishing and requiring labels

A return annotation establishes a fact for the result of a call:

```
public @Persisted Article persist(Article article) {
    return article; // Illustrative trusted boundary, not real persistence.
}
```

The processor trusts the signature. It does not inspect the body to prove that the value was stored. A real application must perform its domain operation before promising the corresponding label.

A parameter annotation requires a fact from callers and supplies that fact as an assumption inside the method:

```
void forward(@Persisted Article article) {
    publishSaved(article);
}
void publishSaved(@Persisted Article article) {}
```

Multiple annotations are conjunctive requirements. Calls to annotated constructor parameters are also checked, as are the individual arguments of expanded varargs.

4.2 Why preservation must be explicit

Without a preservation rule, composition quickly becomes awkward:

```
@Persisted Article persist(Article article) { return article; }
@Validated Article validate(Article article) { return article; }

Article saved = persist(draft);  // Persisted
Article ready = validate(saved); // Validated only
```

The return contract of `validate` says nothing about persistence. Treating every method as preserving every label would be unjustified: a method may change the content or produce a semantically different value.

JSOLV therefore provides `org.jsolv.PreservesTags`:

```
@PreservesTags
public @Validated Article validate(Article article) {
    return article;
}
```

The return receives its explicitly declared labels *and* the labels known for the selected input at that call site. The default selected argument is the first. `@PreservesTags(2)` selects the second using a one-based index. The prototype rejects source declarations with an invalid index, a void return, or a selected varargs parameter.

This is a restricted form of label polymorphism: a validator can preserve any input set without listing all its possible combinations. The rule preserves one input's facts rather than merging unrelated inputs. A validation fact about one object cannot simply become evidence about another input's result.

Preservation is a semantic promise, not a reference-identity test. Returning a different instance can be valid if every input fact remains true for the result. Returning the same instance can be invalid if its contents were changed. In particular, constructing a new article with a changed title does not establish that this new value was persisted merely because its predecessor was persisted. The checker cannot detect such a false preservation promise.

5 A complete workflow example

The following Java file works with the vocabulary in section 3. It uses an in-memory map to illustrate storage, a nonblank-title check, and a Boolean supplied by an external access decision. The access decision is an assumption of this example, not an authentication algorithm. The implementation is intentionally small enough to expose the contracts.

At the four bindings in `run`, the analysis derives:

$$\begin{aligned} L(\texttt{draft}) &= \varnothing, \\ L(\texttt{saved}) &= \{\mathsf{Persisted}\}, \\ L(\texttt{checked}) &= \{\mathsf{Persisted}, \mathsf{Validated}\}, \\ L(\texttt{ready}) &= \{\mathsf{Persisted}, \mathsf{Validated}, \mathsf{Authenticated}\}. \end{aligned}$$

Only the final set satisfies the publication requirement. The same composition can be written as a nested expression:

```
publish(authenticate(validate(persist(draft)), accessGranted));
```

Nested calls are evaluated and checked, rather than treated as opaque syntax.

These alternative calls are rejected:

```
publish(draft);            // Missing all three labels.
publish(validate(saved));  // Missing Authenticated.
```

An unannotated transformation establishes no result labels:

```
Article modify(Article article) { return article; }
// Within a method where ready has all three labels:
Article edited = modify(ready);
publish(edited);           // Rejected, despite this identity-like body.
```

Listing 1: A complete illustrative workflow.

```
package org.jsolv.example.paper;

import java.util.HashMap;
import java.util.Map;
import org.jsolv.PreservesTags;
import org.jsolv.example.tags.Persisted;
import org.jsolv.example.tags.Validated;
import org.jsolv.example.tags.Authenticated;

public final class Workflow {
    public record Article(String title) {}
    private final Map<String, Article> stored = new HashMap<>();

    public @Persisted Article persist(Article article) {
        stored.put(article.title(), article);
        return article;
    }

    @PreservesTags
    public @Validated Article validate(Article article) {
        if (article.title() == null || article.title().isBlank())
            throw new IllegalArgumentException("blank title");
        return article;
    }

    @PreservesTags(1)
    public @Authenticated Article authenticate(
            Article article, boolean accessGranted) {
        if (!accessGranted) throw new SecurityException("denied");
        return article;
    }

    public void publish(
            @Persisted @Validated @Authenticated Article article) {
        System.out.println("Published " + article.title());
    }

    public void run(boolean accessGranted) {
        Article draft = new Article("JSOLV example");
        Article saved = persist(draft);
        Article checked = validate(saved);
        Article ready = authenticate(checked, accessGranted);
        publish(ready);
    }
}
```

This rule permits a transformation that may invalidate facts to omit a preservation promise. It is not an effect-based invalidation of every alias; that distinction matters for mutable objects.

6 Set semantics for the supported local fragment

Let $\mathcal{L}$ be the finite set of fully qualified label identities encountered in a compilation. Two annotations with the same simple spelling in different packages denote different labels. Let B denote local-variable and parameter symbols supplied by the compiler. An abstract environment is

$$\Gamma : B \longrightarrow \mathcal{P}(\mathcal{L}).$$

An omitted symbol has the empty set. The environment records *known positive facts*. A label not in a set is unknown, not proven false. The prototype has no negative-requirement operation.

Write $L_\Gamma(e)$ for the known labels of an expression in the supported fragment, abstracting away state changes made while evaluating subexpressions. For a method f, let $E(f)$ be its declared result labels and $R_i(f)$ its ith parameter requirements. The principal rules are:

$$L_\Gamma(x) = \Gamma(x), \tag{1}$$

$$L_\Gamma(\texttt{new } T(\ldots)) = \varnothing, \tag{2}$$

$$L_\Gamma((T)e) = L_\Gamma(e), \tag{3}$$

$$L_\Gamma(f(e_1, \ldots, e_n)) = E(f) \cup \begin{cases} L_\Gamma(e_k), & \text{if } f \text{ declares preservation of } k, \\ \varnothing, & \text{otherwise.} \end{cases} \tag{4}$$

Parentheses likewise preserve the operand set. Constructor arguments are checked, but construction itself supplies no positive result labels. Call-site checking requires

$$R_i(f) \subseteq L_\Gamma(e_i) \quad \text{for every required argument } i.$$

Missing labels are exactly the set difference $R_i(f) \setminus L_\Gamma(e_i)$.

For assignment, old facts are replaced rather than accumulated:

$$\Gamma' = \Gamma[x \mapsto L_\Gamma(e)] \qquad \text{for } x := e.$$

An annotated local declaration also checks its declared requirements against the initializer. Writing `@Persisted Article forged = draft` cannot manufacture evidence. Nor can a type cast manufacture it.

At an `if` join, after evaluating the condition and both copied branch states, facts common to both alternatives survive:

$$\Gamma_{\text{out}}(x) = \Gamma_{\text{then}}(x) \cap \Gamma_{\text{else}}(x).$$

An omitted else branch preserves the incoming environment. A conditional value expression intersects its alternative result sets as well as joining state. Short-circuit Boolean expressions join the skipped and executed alternatives. The implementation evaluates receivers and arguments in Java order and carries state changes between them; the displayed equations describe their label rules, not a replacement operational semantics for Java.

More facts represent more precise knowledge. Ordering environments by reverse set inclusion makes intersection the usual merge of guaranteed facts. The finite powerset model is straightforward; implementing a sound complete Java analysis over it is a separate problem.

Proposition 1 (Conditional contract consistency in a restricted calculus)**.** *For immutable values, truthful result and preservation contracts, fully modelled expression evaluation, assignments, and branches, the preceding rules transport true positive facts and reject a call whenever its requirements are not derivable.*

The justification is structural: a truthful source supplies true facts; copying and transparent casts retain them; a truthful preservation rule transfers them; and intersection retains only facts supplied by both alternatives. This statement concerns the restricted calculus and its assumptions. It is not a soundness theorem for the prototype over all Java programs, nor proof that an annotated implementation fulfils its contract.

7 Compiler and build architecture

Figure 1 shows the separation between vocabulary production and value-flow checking. The shared annotation library supplies `StaticTag`, `PreservesTags`, and a bundled `Persisted` example. Its vocabulary does not bound the application's generated labels.

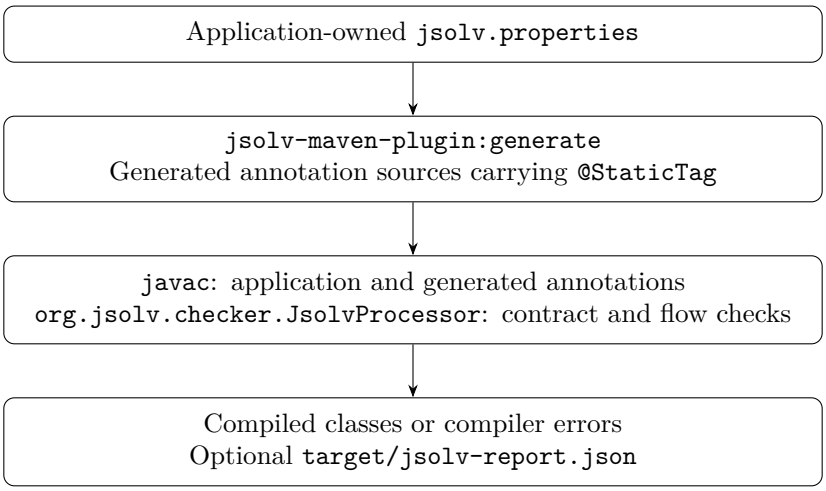


Figure 1: JSOLV in the ordinary Maven compilation lifecycle.

`JsolvProcessor` extends Java's `AbstractProcessor` [3]. It obtains syntax paths and resolved compiler elements through the `Trees` API [4]. A method scanner starts a fresh environment, seeds parameter contracts, and invokes a flow scanner. The flow state maps compiler symbols to immutable sets, avoiding confusion between different variables with the same spelling.

The expression evaluator checks nested calls in initializers, assignments, receivers, and constructor arguments. It reads labels on both type uses and declarations. The processor supports all annotation names so application-defined labels can participate, but returns false from processing so it does not claim annotations needed by other processors. A service-provider file in the checker JAR registers `org.jsolv.checker.JsolvProcessor`.

Generation is deliberately a separate Maven phase, not a first-round side effect of the checker. This ensures label declarations exist before compilation and avoids coupling flow analysis to unresolved generated imports.

7.1 Separate compilation

CLASS retention preserves annotation metadata for consumers. METHOD and PARAMETER targets supplement TYPE_USE metadata. Older javac releases have limitations exposing class-file type annotations to processors, reflected in OpenJDK's later fix [7]. Generated labels placed before a simple return type also annotate the method declaration; the checker consults that declaration. Regression tests exercise library and consumer compilation separately on JDK 17. Purely type-use placements should not be assumed to have the same behaviour on older compilers.

For a multi-module application, generate the vocabulary once in an application-owned labels JAR. Consumers depend on that JAR and put the checker on their annotation-processor path.

They do not need a local copy of the config to recognise those labels. Generating identical fully qualified labels in several modules should be avoided.

8 Maven setup and reproducible use

The repository root contains a Maven reactor with annotation, checker, generator, and demo modules. The `example-app` directory contains an independent consumer. The tested environment uses JDK 17.0.12 and Maven 3.9.11. A JDK is required for compilation and for tests using `ToolProvider.getSystemJavaCompiler()`.

From the repository root:

```
mvn -f pom.xml clean install
mvn -f example-app/pom.xml clean compile
```

These are local development artifacts, version `1.0-SNAPSHOT`; this paper does not assert that they are published on Maven Central. Teams can install them locally or deploy them to their own artifact repository.

A consuming application needs one dependency:

```
<dependency>
  <groupId>org.jsolv</groupId>
  <artifactId>jsolv-annotations</artifactId>
  <version>1.0-SNAPSHOT</version>
</dependency>
```

It also needs the following plugins within `build/plugins`:

```
<plugin>
  <groupId>org.jsolv</groupId>
  <artifactId>jsolv-maven-plugin</artifactId>
  <version>1.0-SNAPSHOT</version>
  <executions>
    <execution><goals><goal>generate</goal></goals></execution>
  </executions>
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.13.0</version>
  <configuration>
    <release>17</release>
    <proc>full</proc>
    <annotationProcessorPaths>
      <path>
        <groupId>org.jsolv</groupId>
        <artifactId>jsolv-checker</artifactId>
        <version>1.0-SNAPSHOT</version>
      </path>
    </annotationProcessorPaths>
    <compilerArgs>
      <arg>-Ajsolv.report=${project.build.directory}/jsolv-report.json</arg>
    </compilerArgs>
  </configuration>
</plugin>
```

The compiler-plugin configuration makes processing explicit [6]. Omitting the report argument

disables reporting without disabling checks. Generator configuration can override `configFile` and `outputDirectory`; the config path is also selectable through `-Djsolv.config=...`. Keep the output dedicated to generated labels.

Use lifecycle commands such as `mvn compile`. Invoking `mvn compiler:compile` directly bypasses the generation phase. IDE users may need to run `mvn generate-sources` and reimport the Maven project before new annotation imports resolve.

9 Diagnostics and inspectable results

When a required set is not available, the processor emits a compiler error at the offending argument. A shortened example is:

```
missing @Authenticated: value lacks required static tags
[org.jsolv.example.tags.Authenticated]
```

The full names preserve identity when labels have the same simple spelling. The prototype reports missing evidence; it does not explain a complete proof history or reconstruct which earlier branch lost a fact.

The optional JSON report has schema version 2, a processor name, and a `checks` array. Each required-argument check records a source URI, line, column, called method's simple name, one-based argument index, required labels, known labels, missing labels, and acceptance. For example:

```
{
  "schema": 2,
  "processor": "org.jsolv.checker.JsolvProcessor",
  "checks": [{
    "source": "file:///example/Workflow.java",
    "line": 42, "column": 17,
    "method": "publish", "argumentIndex": 1,
    "required": ["org.jsolv.example.tags.Authenticated"],
    "known": [],
    "missing": ["org.jsolv.example.tags.Authenticated"],
    "accepted": false
  }]
}
```

The example illustrates the schema rather than a literal transcript of listing 1. Reports include accepted and rejected required-argument checks and are written at the final processing round. They do not enumerate all expressions or all diagnostics. Invalid Java can prevent processing from reaching this stage. An up-to-date Maven compile may skip javac and leave the previous report; `clean compile` produces a fresh compilation.

10 Trust boundaries and limits of enforcement

Label generation does not generate the domain operation. A truthful `@Validated` method must actually establish the application's definition of validation. An `Authenticated` label is not intrinsically tied to a principal, resource, policy, or time. Such context must be represented in the application contract; parameterised labels are not implemented.

The prototype checks calls against contracts but does not verify return statements against promised result labels, infer method summaries, or check override compatibility. A false annotation can therefore cause acceptance of an invalid domain workflow. Auditing introducing and preserving methods is an essential part of adoption.

10.1 Bindings are not a heap analysis

Copying a binding copies its tracked facts. Reassigning one binding leaves another binding's tracked facts intact. This models value provenance for local variables, not mutable object state shared by references. Consider:

```
Document saved = persist(document);
Document alias = saved;
alias.setContents("different content");
publishSaved(saved);
```

If persistence depends on the current contents, the old fact may now be false. The checker does not automatically revoke it after the mutating call. Immutable data and reviewed transformation contracts fit the prototype much better than arbitrary mutable protocols. Immutability alone also does not make an incorrect preservation annotation true.

10.2 Control flow and unsupported constructs

`if` and conditional-expression joins use intersections. Loops, try/catch/finally, and switch statements conservatively forget facts for assigned variables and do not establish new facts after the construct; switch cases and exception branches are inspected separately. The processor does not build a complete control-flow graph or compute loop fixed points. Early returns, break/continue, exceptions, and reachability do not prune joins. Safe programs may therefore be rejected.

Fields and array elements supply no known label facts. Generic element flow, receiver contracts, method references, streams, and asynchronous callbacks are not modelled. Lambda bodies start separate state with parameter contracts and unknown captured facts. Switch expressions supply no result facts and clear tracked state. An annotated array passed as a single varargs argument is not modelled like a sequence of individually checked values. Initializers outside method bodies are outside the current scans.

Conservative treatment of some constructs does not establish global soundness. The unmodelled effects and mutable-alias example already preclude such a claim. The actual guarantee is a useful collection of supported contract and flow checks, not a security proof for arbitrary Java.

11 Relationship to the broader proposal and other techniques

Table 1 distinguishes what JSOLV implements from the wider static property proposal. This separation avoids interpreting a convenient annotation syntax as a claim that difficult analysis problems have been solved.

Table 1: Scope of the present implementation.

Concept	JSOLV status
User-defined positive atomic labels	Implemented through configuration and generated annotations.
Value-flow requirements and result contracts	Implemented for the documented local fragment; result contracts are trusted.
Independent label composition	Implemented by sets and explicit single-input preservation.
Negative facts and requirements	Not implemented; absence means unknown.
Validator Boolean branch refinement	Not implemented; call result contracts establish facts on returned values.

Concept	JSOLV status
Alias-aware or effect-based revocation	Not implemented.
Full CFG and loop fixed points	Not implemented.
Parameterised labels and implications	Possible extensions, not configuration features today.
Proof-producing or verified validators	Possible extensions; current boundaries are trusted.

JSOLV overlaps with pluggable type qualifiers, typestate, and taint analysis. The Checker Framework demonstrates a substantially richer approach to pluggable Java checking [8]; JSOLV does not claim equivalent coverage or qualifier polymorphism. Its set-based facts are independent rather than one enumerated protocol state, and its core vocabulary is domain-neutral. It can resemble an opaque refinement layer, but it has no general predicates, solver, or proof language.

Compared with wrappers, labels preserve the ordinary API and avoid introducing combination-specific classes. Wrappers can provide stronger nominal separation and carefully controlled construction, so the mechanisms are complementary. The architectural distinction advocated by the semantic-categories paper is realised here without changing Java's parser or runtime representation.

12 Regression evidence and evaluation boundaries

The current regression suite contains 24 tests: three generator tests, four original persistence-checker tests, and seventeen configurable-label tests. The renamed JSOLV build passes these tests in the stated JDK/Maven environment. The independent application exercises actual Maven goal binding, service-based processor discovery, generated imports, and JSON reporting.

The tests cover configuration validation, owned-file cleanup, unchanged output, multiple simultaneous requirements, explicit and selected-argument preservation, loss after unannotated transformation, parameter forwarding, branch intersection, casts, scope identity, nested calls, constructor arguments, expanded varargs, selected control-flow cases, reports, and separately compiled libraries. One test adds a new configuration label, generates its annotation, compiles a valid use, and rejects a missing-label use without changing the checker. Another distinguishes equally spelled labels from different packages.

These are regression observations, not an empirical demonstration of defect reduction, acceptable false-positive rates, or scalability. No developer study, large-codebase benchmark, or compilation-overhead measurement is reported. The next evaluation should measure annotation burden, unsupported-pattern frequency, compilation cost, and whether diagnostics help developers repair real workflows without adding incorrect trust contracts.

13 Adoption and future development

A useful adoption order is to identify a few meaningful sinks, annotate their requirements, and review the methods that genuinely establish those facts. Then introduce preservation only where every existing fact remains true for the result. Prefer immutable values, narrow contracts, and application-owned label packages. In a shared library, publish the generated vocabulary once.

Future analysis work should prioritise a complete control-flow representation, effect-aware invalidation, override checks, and verified result contracts before claiming stronger guarantees. Selective preservation would help transformations that retain some facts while invalidating others. Contextual or parameterised labels could express authorisation for a particular resource,

but require a precise identity model. Label implications should likewise have explicit, bounded semantics rather than unrestricted rules hidden in configuration.

The two-key vocabulary and marker protocol need not change as analysis improves. Keeping application naming separate from compiler logic lets the tool develop without transferring checker-maintenance responsibilities to application teams.

14 Conclusion

JSOLV makes a useful subset of static property tagging available in ordinary Java. Application teams name their facts in a small configuration file, generated annotations expose those facts in signatures, and a generic checker enforces supported value-flow requirements during compilation. Explicit preservation allows independent workflow stages to compose without a family of nominal types or a registry embedded in checker code.

The design's effectiveness comes from a clear division of responsibilities: applications own vocabulary and truthful contracts; generation supplies ordinary Java declarations; analysis transports sets of known facts; diagnostics expose missing evidence. Its limits are equally explicit. The implemented tool is a practical workflow checker with trusted boundaries, and a foundation for stronger analysis, rather than a completed solution to arbitrary object-state verification.

References

[1] Sambuddha Majumder and Jayanta Majumder. *User-Defined Static Property Tags for Mainstream Programming Languages: A Lightweight Flow-Sensitive Verification Layer*. Companion manuscript, `static_property_tags.tex`.

[2] Jayanta Majumder and Sambuddha Majumder. *Explicit Semantic Categories for More Legible Mainstream Programming Languages*. Companion manuscript, `semantic_roles_beyond_class.tex`.

[3] Oracle. *Java SE 17 API: AbstractProcessor*. `https://docs.oracle.com/en/java/javase/17/docs/api/java.compiler/javax/annotation/processing/AbstractProcessor.html`

[4] Oracle. *Java SE 17 API: Trees*. `https://docs.oracle.com/en/java/javase/17/docs/api/jdk.compiler/com/sun/source/util/Trees.html`

[5] Apache Maven. *Introduction to the Build Lifecycle*. `https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html`

[6] Apache Maven. *Maven Compiler Plugin 3.13.0: compiler:compile*. `https://maven.apache.org/plugins-archives/maven-compiler-plugin-3.13.0/compile-mojo.html`

[7] OpenJDK. *Type annotations are not visible to javac plugins across compilation boundaries*, JDK update discussion, March 2025. `https://mail.openjdk.org/pipermail/jdk-updates-dev/2025-March/041891.html`

[8] The Checker Framework developers. *The Checker Framework*. `https://checkerframework.org/`